

Tutorial final

CRUD de E-commerce com Angular, Express, Postman e GitHub

Passo a passo para validar localmente, publicar o fork publico e enviar a URL ao AVA.

Arquivo gerado a partir de docs/TUTORIAL-FINAL-GITHUB-AVA.md

Este tutorial conclui a parte que ainda ficou pendente: conferir visualmente a aplicação no seu computador, criar o fork público, publicar o código no GitHub e enviar a URL ao AVA.

O objetivo da atividade é publicar o código-fonte completo em um repositório GitHub público. A aplicação continua sendo uma API local com frontend local. Não é necessário publicar a API em servidor externo, configurar GitHub Pages, contratar hospedagem ou transformar o projeto em site online.

1. O que precisa estar pronto no final

Ao terminar, você deve ter:

Item	Resultado esperado
Fork público	Um repositório seu no GitHub, público, baseado em HevelinFR/angular-crud-api
Código atualizado	Pastas frontend, backend, postman, docs, scripts e arquivos de configuração no seu repositório
Teste local	Frontend em http://localhost:4200 e API em http://localhost:3000/api/products funcionando no seu computador
Evidência de API	Coleção Postman importada ou resultado Newman em docs/evidencias/resultado-postman.json
Envio AVA	URL pública do seu repositório enviada na atividade correta

Nesta sessão, a validação automatizada passou:

Validação	Resultado
API + Postman/Newman	13 requisitos, 27 verificações, 0 falhas
Frontend Angular	3 arquivos de teste, 13 testes, 0 falhas
Build Angular	Concluído; houve apenas aviso de tamanho inicial

A publicação no GitHub não foi executada automaticamente porque a ferramenta disponível nesta sessão não cria fork. Essa parte precisa ser feita no navegador da sua conta GitHub.

2. Instalar e conferir as ferramentas

Use preferencialmente o Prompt de Comando do Windows ou o terminal Command Prompt dentro do VS Code. O PowerShell também funciona, mas pode bloquear scripts `npm.ps1` em alguns computadores.

Instale ou confira:

Ferramenta	Para que serve	Link oficial
Node.js 24 LTS	Executar backend, Angular e npm	Node.js Downloads
Git for Windows	Clonar, versionar e enviar ao GitHub	Git for Windows
Visual Studio Code	Editar e abrir terminais do projeto	VS Code
Postman Desktop	Testar os endpoints da API local	Postman Downloads

Depois de instalar, feche e abra o terminal. Confira:

```
node -v
npm -v
git --version
```

O projeto foi validado com Node 24.19.0 e npm 11.9.0. O package.json aceita Node ^22.22.3 || ^24.15.0 || >=26.0.0. Para simplificar, use Node 24 LTS, versao 24.15.0 ou superior dentro da linha 24.

O Angular CLI global nao e obrigatorio para este projeto. Apos instalar o frontend, voce pode conferir o CLI local assim:

```
npm exec --prefix frontend -- ng version
```

Se o professor pedir explicitamente o comando ng -v, instale o CLI global:

```
npm install -g @angular/cli  
ng version
```

3. Criar o fork publico no GitHub

- 1 Entre na sua conta em github.com.
- 2 Abra o repositorio base: <https://github.com/HevelinFR/angular-crud-api>.
- 3 Clique em Fork.
- 4 Em Owner, selecione o seu perfil.
- 5 Mantenha o nome angular-crud-api, se estiver disponivel.
- 6 Clique em Create fork.

A documentacao oficial do GitHub descreve esse fluxo em [Fork a repository](#).

Depois da criacao, o endereco do seu fork ficara parecido com:

```
https://github.com/SEU-USUARIO/angular-crud-api
```

Se voce usar a conta identificada nesta sessao, o endereco esperado apos criar o fork deve ficar parecido com:

```
https://github.com/edgarmagalhaes007-dotcom/angular-crud-api
```

Nao envie ao AVA a URL da professora. Envie a URL do repositorio que esta no seu perfil.

4. Clonar o seu fork no computador

Crie uma pasta para projetos. Exemplo:

```
cd %USERPROFILE%  
mkdir Projetos  
cd Projetos
```

No GitHub, abra o seu fork, clique em Code e copie a URL HTTPS. Depois clone:

```
git clone https://github.com/SEU-USUARIO/angular-crud-api.git  
cd angular-crud-api
```

Confira se o clone aponta para o seu fork:

```
git remote -v
```

O resultado deve mostrar origin apontando para <https://github.com/SEU-USUARIO/angular-crud-api.git>.

Se aparecer HevelinFR/angular-crud-api, voce clonou o repositorio original, nao o seu fork. Nesse caso, entre na pasta correta do seu fork ou ajuste o remoto:

```
git remote set-url origin https://github.com/SEU-USUARIO/angular-crud-api.git
```

Use esse comando somente se tiver certeza de que a pasta atual e o projeto certo.

5. Copiar o pacote final para dentro do fork

Extraia o arquivo final da entrega em uma pasta separada, por exemplo em Downloads.

Dentro do ZIP existe uma pasta do projeto chamada angular-crud-api. Copie o conteudo dessa pasta para dentro da pasta clonada do seu fork.

O resultado correto deve ser assim:

```
angular-crud-api/  
  backend/  
  frontend/  
  postman/  
  docs/  
  scripts/  
  package.json  
  README.md
```

Evite este erro:

```
angular-crud-api/  
  angular-crud-api/  
    backend/  
    frontend/
```

Se isso acontecer, mova o conteudo da pasta interna para a raiz do clone.

Preserve a pasta Git criada pelo clone. Ela e invisivel no Windows por padrao e se chama `.git`. Nao execute `git init` dentro do projeto ja clonado.

Depois de copiar, confira:

```
git status
```

E esperado aparecerem arquivos modificados ou novos. Nao devem aparecer `node_modules`, `.angular`, `dist` ou arquivos de senha.

6. Instalar dependencias

Abra o projeto no VS Code:

```
code .
```

No terminal do VS Code, estando na raiz do projeto, execute:

```
npm ci  
npm ci --prefix backend
```

```
npm ci --prefix frontend
```

Esses tres comandos usam os arquivos `package-lock.json` e deixam o ambiente mais previsivel.

Se aparecer erro dizendo que `package.json` nao foi encontrado, voce esta na pasta errada. Volte para a raiz do projeto, onde ficam `backend`, `frontend`, `postman`, `docs` e o `package.json` principal.

7. Rodar a validacao automatizada

Antes de iniciar os servidores manualmente, rode os testes. A API deve estar parada para o teste automatizado usar a porta 3000.

Na raiz do projeto:

```
npm run test:api
```

Resultado esperado:

```
1  Mensagem de persistencia aprovada.
1  Newman executando a colecao.
1  13 requests executadas.
1  27 assertions.
1  0 failures.
```

O arquivo de resultado fica em:

```
docs/evidencias/resultado-postman.json
```

Depois rode:

```
npm run test:frontend
npm run build
```

Resultado esperado:

```
1  3 arquivos de teste aprovados.
1  13 testes aprovados.
1  Build concluido.
1  Pode aparecer aviso de tamanho inicial do bundle, mas nao erro.
```

Se o build terminar com `Application bundle generation complete`, ele esta ok para esta entrega.

8. Iniciar a API e o frontend

Agora abra dois terminais no VS Code.

Terminal 1, na raiz do projeto:

```
npm run dev --prefix backend
```

Deixe esse terminal aberto. A API deve ficar em:

`http://localhost:3000/api/products`

Terminal 2, também na raiz do projeto:

`npm start --prefix frontend`

A interface deve abrir ou ficar disponível em:

`http://localhost:4200`

Abra `http://localhost:4200` no navegador. Não abra a interface na porta 3000; a porta 3000 é da API.

9. Conferir visualmente no navegador

Use este roteiro para garantir que a aplicação está funcionando de ponta a ponta.

- 1 Abra `http://localhost:4200`.
- 2 A lista de produtos deve carregar com os produtos vindos da API.
- 3 Clique em Novo Produto.
- 4 Cadastre um produto de teste:

Nome: Controle Gamer - Validacao

Descricao: Controle sem fio para computador e console.

Categoria: Jogos

Preco: 249.90

Imagem: deixe em branco ou use uma URL `http/https` valida

- 1 Clique em salvar. A aplicação deve mostrar mensagem de sucesso e voltar para a lista.
- 2 Atualize a página com F5. O produto cadastrado deve continuar aparecendo.
- 3 Clique em Editar no produto criado.
- 4 Altere o preço para 199.90 e salve.
- 5 Confira se a lista voltou com o novo valor.
- 6 Clique em Excluir e cancele a confirmação. O produto deve permanecer.
- 7 Clique novamente em Excluir e confirme. O produto deve desaparecer.

Teste também o comportamento de erro:

- 1 Com a lista aberta, pare apenas o backend pressionando `Ctrl+C` no Terminal 1.
- 2 Recarregue a página.
- 3 A interface deve mostrar uma mensagem de erro de conexão e o botão Tentar novamente.
- 4 Reinicie o backend.
- 5 Clique em Tentar novamente.

Teste o formulário:

- 1 Entre em Novo Produto.
- 2 Preencha alguns dados.
- 3 Pare a API.

- 4 Tente salvar.
- 5 A aplicacao deve manter os dados no formulario e mostrar mensagem de erro.

Teste um produto inexistente:

`http://localhost:4200/produtos/editar/999999999`

A aplicacao deve informar que o produto nao foi encontrado ou impedir a edicao invalida.

10. Conferir no Postman

Com a API ligada, abra o Postman Desktop.

- 1 Clique em Import.
- 2 Importe `postman/TechStore-CRUD.postman_collection.json`.
- 3 Importe `postman/Local.postman_environment.json`.
- 4 Selecione o ambiente TechStore Local.
- 5 Confira a variavel `baseUrl`:

`http://localhost:3000`

- 1 Abra a colecao TechStore - CRUD de Produtos.
- 2 Clique em Run collection.
- 3 Execute uma iteracao, mantendo a ordem original.

Resultado esperado:

Campo	Esperado
Requests	13
Assertions	27
Failures	0

A colecao cria, consulta, atualiza e exclui um produto de teste. O ID e capturado automaticamente pelo Postman. Nao use ID fixo nem altere a lista manualmente durante a execucao da colecao.

Se estiver usando Postman Web, talvez seja necessario instalar ou ativar o Desktop Agent para acessar localhost. O Postman Desktop costuma ser o caminho mais simples.

11. Tirar evidencias opcionais

A atividade pede principalmente a URL do repositorio. Mesmo assim, e recomendavel incluir evidencias.

Sugestao de prints:

Print	O que mostrar
Lista de produtos	Interface em localhost:4200 com produtos carregados
Cadastro	Produto novo salvo e aparecendo na lista
Edicao	Produto com preco alterado

Print	O que mostrar
Exclusao	Lista sem o produto removido
Postman	Runner com 13 requests e 0 failures
GitHub	Repositorio publico com README aparecendo

Voce pode salvar os prints em:

docs/evidencias/prints/

No Windows, use Win + Shift + S, cole no Paint ou em outro editor e salve como PNG. Evite mostrar senhas, tokens, e-mails privados ou dados pessoais desnecessarios.

12. Publicar no GitHub

Depois de testar, pare os servidores com Ctrl+C nos dois terminais.

Confira o status:

```
git status
```

Confira se nao existem pastas pesadas ou temporarias:

```
git status --short
```

Nao devem entrar no commit:

```
node_modules/
.angular/
dist/
coverage/
*.log
.env
```

Esses itens ja estao cobertos pelo .gitignore.

Configure sua identidade Git se ainda nao tiver feito:

```
git config --global user.name "SEU NOME"
git config --global user.email "SEU EMAIL DO GITHUB"
```

Se preferir, use o e-mail noreply exibido pelo proprio GitHub em Settings > Emails. Nao copie um e-mail de exemplo sem trocar pelos seus dados.

Adicione e envie:

```
git add .
git diff --cached --stat
git commit -m "Conclui integracao CRUD de produtos"
git push origin main
```

Durante o git push, o GitHub pode abrir uma tela de login no navegador por meio do Git Credential Manager. Use o login normal do GitHub. Nao digite sua senha do GitHub como senha HTTPS no terminal; o GitHub usa autenticacao por navegador ou token.

Se o seu fork estiver na branch master, substitua main por master:

```
git branch --show-current
git push origin master
```

Nao use `git push --force` para esta entrega.

13. Conferir o repositorio publico

Depois do push:

- 1 Abra o repositorio no GitHub.
- 2 Confira se o README novo aparece na pagina inicial.
- 3 Verifique se existem as pastas backend, frontend, postman, docs e scripts.
- 4 Confira se docs/evidencias/resultado-postman.json esta no repositorio.
- 5 Confira se o repositorio aparece como Public.
- 6 Abra a URL em uma janela anonima ou outro navegador sem login.

Se a janela anonima abrir o repositorio, o tutor tambem deve conseguir acessar.

Observacao importante: forks de repositorios publicos normalmente ficam publicos dentro da rede do GitHub. Se voce criou um repositorio novo em vez de fork, a visibilidade fica em Settings > General > Danger Zone > Change repository visibility.

14. Enviar a URL ao AVA

No AVA:

- 1 Abra a disciplina correta.
- 2 Entre na atividade de entrega do CRUD/API local.
- 3 Cole a URL publica do seu repositorio.
- 4 Anexe prints ou a colecao Postman somente se o AVA tiver campo para isso ou se o professor pedir.
- 5 Salve.
- 6 Confirme se aparece status de envio concluido, enviado ou submetido.

A URL correta deve ser semelhante a:

```
https://github.com/SEU-USUARIO/angular-crud-api
```

Nao envie:

```
http://localhost:4200
http://localhost:3000/api/products
https://github.com/HevelinFR/angular-crud-api
```

localhost so funciona no seu computador. A URL do AVA precisa ser a URL publica do codigo no GitHub.

15. Checklist final antes de enviar

Marque antes do envio:

Conferencia	OK
Node, npm e Git instalados	
Fork criado na sua conta	
Codigo final copiado para o fork	
npm ci executado na raiz, backend e frontend	
npm run test:api passou	
npm run test:frontend passou	
npm run build concluiu	
Interface abriu em localhost:4200	
API respondeu em localhost:3000/api/products	
Cadastro funcionou visualmente	
Edicao funcionou visualmente	
Exclusao funcionou visualmente	
Colecao Postman executou sem falhas	
Commit criado	
Push enviado ao GitHub	
Repositorio esta publico	
URL publica enviada no AVA	

16. Problemas comuns

Problema	Como resolver
node nao reconhecido	Reinstale Node 24 LTS, marque a opcao de adicionar ao PATH e reabra o terminal
npm.ps1 cannot be loaded	Use Prompt de Comando no VS Code ou execute npm.cmd
ng nao reconhecido	Use npm start --prefix frontend ou npm exec --prefix frontend -- ng version
ENOENT package.json	Voce esta na pasta errada; volte para a raiz do projeto
Porta 3000 ocupada	Pare a API aberta anteriormente com Ctrl+C
Cannot GET / na porta 3000	A API fica em /api/products, nao na raiz /
Angular nao lista produtos	Confirme se backend e frontend estao ligados ao mesmo tempo
Postman falha em localhost	Use Postman Desktop ou Desktop Agent
400 Bad Request	Revise campos obrigatorios e preco maior que zero
404 Not Found	Verifique o ID usado e a rota
git push negado	Confira se origin aponta para o seu fork e faca login no GitHub
Repositorio privado	Altere a visibilidade para publico ou recrie como fork publico
Pasta duplicada angular-crud-api/angular-crud-api	Mova o conteudo da pasta interna para a raiz do clone

17. O que este projeto demonstra

O projeto atende ao objetivo da atividade porque:

Capacidade	Como aparece no projeto
Reconhecer servicos do servidor	Rotas Express em backend/src/server.js
Reconhecer especificacoes	README, Postman e contrato da API documentam rotas, corpos e respostas

Capacidade	Como aparece no projeto
Usar serviços do servidor	ProdutoService chama GET, POST, PUT e DELETE
Tratar falhas do servidor	Componentes exibem erros de conexão, 400, 404 e falhas inesperadas
Integrar framework e linguagem	Angular usa HttpClient, rotas, componentes, formulários reativos e Bootstrap
Refletir interações do usuário	Cadastro, edição, listagem e exclusão alteram dados pela API local

Arquivos principais:

Arquivo	Papel
frontend/src/app/produtos/produto.service.ts	Chamadas HTTP do CRUD
frontend/src/app/produtos/produto-formulario/	Cadastro e atualização
frontend/src/app/produtos/lista-produtos/	Consulta e exclusão
frontend/src/app/produtos/erro-api.ts	Mensagens de erro exibidas ao usuário
backend/src/server.js	API local Express
backend/data/produtos.json	Dados persistidos localmente
postman/TechStore-CRUD.postman_collection.json	Roteiro de teste dos endpoints

18. Referências usadas

- 1 Repositório base: [HevelinFR/angular-crud-api](#)
- 1 Criação de fork: [GitHub Docs - Fork a repository](#)
- 1 Adicionar arquivos no GitHub: [GitHub Docs - Adding a file to a repository](#)
- 1 Autenticação no GitHub: [GitHub Docs - About authentication to GitHub](#)
- 1 Compatibilidade Angular: [Angular - Version compatibility](#)
- 1 Node.js: [Node.js Downloads](#)
- 1 Git for Windows: [Git Downloads for Windows](#)
- 1 VS Code: [Visual Studio Code Download](#)
- 1 Postman: [Postman Downloads](#)